Week 8 Assignment

Repository Link:
https://github.com/Hughes-Tyler/AnalyticsCapstone

Report Link:
https://github.com/Hughes-Tyler/AnalyticsCapstone/blob/main/notebooks/main.ipynb

This week we trained our third modeling method, a Multi-Layer Perceptron (MLP) neural network using scikit-learn's MLPClassifier, on the same processed startup dataset and train/validation split from Weeks 6 and 7. As in previous weeks, we tested three hyperparameter variations with increasing complexity, evaluated their training and validation performance, and selected the variation that generalized best. We also compared our winning neural network against Week 7's Gradient Boosting model to determine whether the added complexity improved our ability to predict startup outcomes.

We chose an MLP neural network because it approaches classification differently from the tree-based models we previously tested. Our dataset contains four heavily imbalanced outcomes: operating (81.7%), acquired (8.3%), closed (8.0%), and IPO (2.0%). Random Forest and Gradient Boosting make predictions through combinations of decision trees, while an MLP uses interconnected layers of neurons to learn nonlinear relationships between features. This allows the model to identify more complicated interactions across our 51 input features, including the five PCA components and 46 binary indicators. For example, the network can learn relationships involving funding patterns, company characteristics, and geographic information simultaneously. However, this flexibility also introduces additional parameters, requires feature scaling, and makes the model considerably harder to interpret than our previous methods.

Unlike Random Forest and Gradient Boosting, MLPClassifier does not directly support the class-weighting approach we used previously. To address the class imbalance, we applied random oversampling to the training data, duplicating minority-class observations until each of the four classes contained 24,330 observations. This increased our training dataset from 29,782 to 97,320 rows. Importantly, we only oversampled the training data, leaving the 9,928 validation observations unchanged. We also used StandardScaler to standardize the features, fitting the scaler on the training data and applying the same transformation to validation.

We evaluated three hyperparameters that influence network complexity and overfitting. The hidden_layer_sizes parameter determines the number and size of hidden layers, controlling how many nonlinear relationships the model can learn. The alpha parameter controls L2 regularization, which penalizes large weights and helps prevent overfitting. Finally, learning_rate_init determines how quickly the model updates its weights during training. We varied these parameters together to test how increasing model capacity and reducing regularization affected generalization. Our three variations were:

- V1 – Small: hidden_layer_sizes=(16,), alpha=0.1, learning_rate_init=0.001. A simple network with one hidden layer and strong regularization, expected to underfit.
- V2 – Moderate: hidden_layer_sizes=(64, 32), alpha=0.001, learning_rate_init=0.001. A moderately complex network with two hidden layers and lighter regularization, intended to balance model flexibility and generalization.
- V3 – Large: hidden_layer_sizes=(256, 128, 64), alpha=0.000001, learning_rate_init=0.01. A much larger network with three hidden layers, minimal regularization, and faster learning, expected to have the greatest risk of overfitting.

All three variations used ReLU activation, the Adam optimizer, a batch size of 256, a maximum of 300 training iterations, and random_state=42. Early stopping was disabled to maintain consistent training settings across variations. We trained each network, generated predictions on both the original training and validation datasets, and saved the three trained models along with a combined week8_predictions.csv file containing 119,130 predictions. Evaluating against the original training observations, rather than their oversampled duplicates, also allowed us to maintain consistency with the previous weeks.

We continued using macro F1, per-class recall, and per-class precision as our primary evaluation metrics. Macro F1 remained our main selection criterion because it gives equal importance to all four classes regardless of their frequency. This is especially important because a model predicting every startup as operating would achieve approximately 81.7% accuracy while completely failing to identify acquisitions, closures, and IPOs. Recall measures how many companies belonging to each outcome the model correctly identifies, while precision measures how often its predictions for a particular class are correct. Together, these metrics allow us to evaluate whether improvements in minority-class detection come at the cost of excessive false positives.

Our results showed that increasing network complexity substantially improved training performance but produced very limited improvements on validation. Training macro F1 increased from 0.419 for V1 to 0.656 for V2 and 0.760 for V3. However, validation macro F1 increased only slightly, from 0.369 to 0.377 and finally 0.383. Consequently, the difference between training and validation macro F1 grew from 0.050 for V1 to 0.279 for V2 and 0.377 for V3. This suggests that the additional network complexity primarily helped the models fit the training observations rather than generalize to unseen companies.

V1 showed the strongest signs of underfitting. Its training and validation scores were relatively close, indicating that the network was not memorizing the training data, but its limited capacity prevented it from learning sufficiently accurate decision boundaries. Interestingly, V1 achieved the highest validation recall for all three minority classes: 0.546 for acquired, 0.480 for closed, and 0.611 for IPO. However, this came at the cost of very low precision, particularly for IPO at 0.112. In other words, the model identified many actual IPO companies but incorrectly classified a large number of other companies as IPOs.

V2 improved training performance considerably, but these gains did not translate into comparable validation improvements. Its training IPO recall reached 0.995, while validation IPO recall dropped to 0.323. Similar declines occurred for acquired and closed companies, indicating that the model had begun learning patterns specific to the training observations. V3 demonstrated the strongest overfitting pattern, with a training macro F1 of 0.760 compared with only 0.383 on validation. Its validation recall for operating companies increased to 0.745, but recall for acquired, closed, and IPO companies fell to 0.363, 0.317, and 0.303, respectively. This suggests that the larger network increasingly favored the majority class while losing its ability to identify less common outcomes.

Despite this overfitting, we selected V3 (Large) as our winning Week 8 variation because it achieved the highest validation macro F1 of 0.383, compared with 0.377 for V2 and 0.369 for V1. It also achieved the highest validation accuracy at 67.0% and the highest acquired-class precision at 0.254. However, its advantage over the other variations was small, and its large training-validation gap suggests that its additional complexity did not provide a meaningful improvement in generalization.

Finally, we compared our winning MLP against Week 7's Gradient Boosting V2 using the results calculated in the notebook. Gradient Boosting achieved a validation macro F1 of 0.431,

compared with 0.383 for the MLP. It also performed better on recall for every minority class: acquired (0.552 vs. 0.363), closed (0.448 vs. 0.317), and IPO (0.419 vs. 0.303). The MLP performed slightly better on operating recall (0.745 vs. 0.702) and overall accuracy (0.670 vs. 0.663), but these improvements were not sufficient to offset its weaker minority-class performance.

In conclusion, Week 8 demonstrated that a more complex modeling approach does not necessarily produce better predictions. Although the neural network could learn increasingly complicated relationships in the training data, its validation performance remained relatively weak, particularly for the minority classes that are most important to our project. The oversampling strategy may also have contributed to overfitting by repeatedly presenting the same minority-class observations during training. While V3 was the best-performing neural network variation, it did not outperform Gradient Boosting V2. Therefore, we retained Week 7's Gradient Boosting model as our strongest overall classifier and did not carry the Week 8 MLP forward for final test-set evaluation.